


UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

**Avance 06/10/2026 - CodeMichi**

**Presentado a:**

Grupos de Investigación Multimedia Interactiva

**Presentado por:**

Yader Ibraldo Quiroga Torres – 20222020034

Jhonathan De La Torre - 20222020033

Facultad de Ingeniería

Ingeniería de sistemas

6 de Octubre 2026

# Reporte de Avances — CodeMichi

**Proyecto:** Juego educativo de programación por bloques (gato minero 2.5D)

## 1. Objetivo cumplido

Logramos tomar la funcionalidad del prototipo (donde el personaje se mueve por casillas usando únicamente bloques de programación: avanzar, girar, activar) y la integramos exitosamente al proyecto visual que también se trabajó simultáneamente, este proyecto visual cuenta con estilo artístico de personaje pixel art en entorno 3d, para este apartado trabajamos en el desarrollo del gato animado, el movimiento con flechas y la grilla para poder probar animaciones de movimiento básico, luego le pusimos la funcionalidad de dicho prototipo; como resultado ahora el gato se mueve por bloques y puede resolver el nivel picando el mineral que sirve como meta, todo con una interfaz fluida que no interfiere con la experiencia de juego.

## 2. Avances detallados por archivo

| Archivo / Componente | Descripción / Funcionalidad |
|---|---|
| `Scripts/player.gd` | • Implementa la API `execute_command` (`"avanzar"`, `"girar_izq"`, `"girar_der"`, `"activar"`/`"picar"`), totalmente compatible con el ejecutor de programas.<br>• Añade movimiento discreto por casillas de 2.0 m con transición (tween) suave usando animaciones existentes (Caminar / Inactivo / Picar).<br>• Integra orientación de 4 direcciones (N/E/S/O) sincronizada con el sprite.<br>• Incluye modo programa (`program_running`) para bloquear el control manual durante la ejecución.<br>• Válida colisiones físicas (`test_move`), solo muros y rocas reales frenan al gato (con rebote visual y registro de fallo).<br>• El comando `activar` equivale a picar, reproduciendo la animación y removiendo la celda objetivo.<br>• Implementa guardado y reinicio de estado (`save_program_start` / `reset_program_state`).<br>• Conserva la barra espaciadora para picar manualmente. |

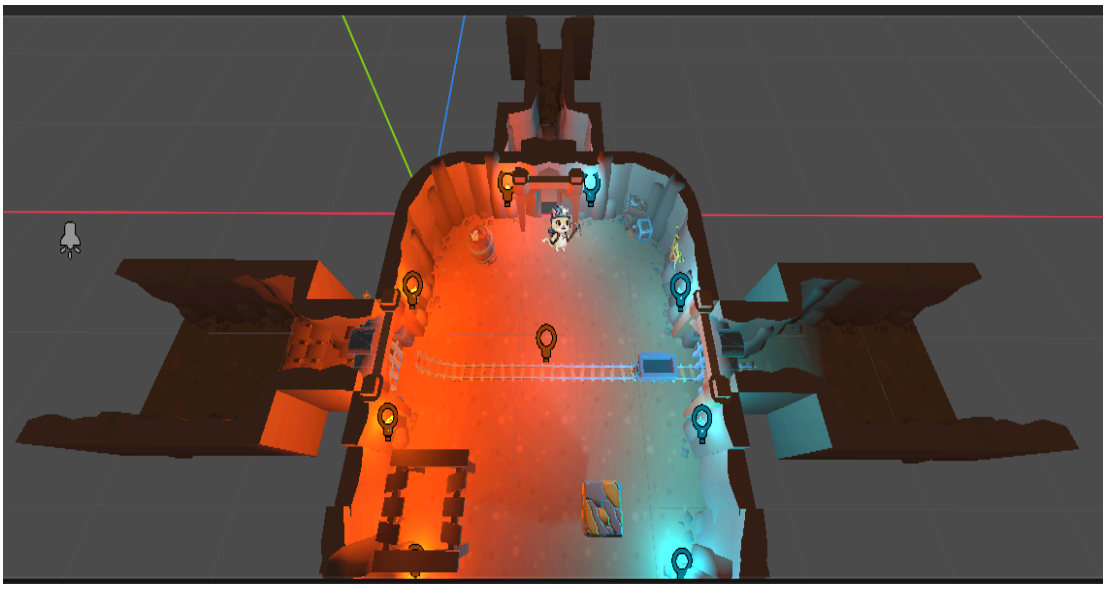
| Archivo / Componente | Descripción / Funcionalidad |
| --- | --- |
| | • Agrega indicador de orientación (flecha amarilla y letra N/E/S/O) y marcador gris de casilla en el suelo. |
| `Scripts/CommandRunner.gd` | Adaptación 1:1 del prototipo de jugabilidad presentado en la primera socialización del proyecto, ejecuta la secuencia de bloques paso a paso sobre el jugador, emitiendo las señales `step_started`, `step_finished` y `finished(success, failed_index)`. |
| `Scripts/Block.gdScenes/Block.tscn` | Bloques arrastrables organizados por color según categoría, con resaltado de ejecución (activo/ok/fallo) y forma de rompecabezas, clic derecho para borrar y doble clic en paleta para agregar directo. |
| `Scripts/Workspace.gd` | Lista vertical del programa, acepta drag-and-drop, permite reordenar bloques, tiene scroll interno y emite la señal `program_changed` ante cualquier cambio. |
| `Scripts/CodeUI.gd` | Orquestador no invasivo que reemplaza al `Main.gd` del prototipo adaptándolo al entorno 3D:<br>• Panel lateral colapsable (tecla `C`), con paleta compacta, workspace, vista de pseudocódigo, botones de control y mensajes.<br>• Atajos: `C` (abrir/cerrar), `F5` / `Ctrl+Enter` (ejecutar), `R` (reiniciar/detener).<br>• Detección automática del jugador, GridMap y celda del mineral.<br>• Condición de victoria al picar el mineral ("¡Éxito!") y diagnóstico exacto en caso de fallo. |
| `Scripts/Meta Beacon.gd` Nodo `Meta` en `Mapa1.tscn` | Marca visual para el objetivo generada por código (columna de luz, anillo ajustado y etiqueta "¡META!"). Se centra en el mineral y cambia a verde ("¡ABIERTA!") al ganar. |
| `Scenes/Mapa1.tscn` | Mantiene el nivel, gato y grilla intactos. Únicamente se integraron los nodos `CodeUI` y `Meta`. |
| `Shaders/ShaderMallaNeon.gdshader` | Nuevo parámetro `grid_offset` para alinear perfectamente la grilla de neón con las celdas de 2 metros del juego. |

## 3. Gameplay resultante

- **Control exclusivo:** El gato se mueve **únicamente a través de los bloques** (cumpliendo el objetivo del prototipo), las teclas de movimiento libre quedaron desactivadas y comentadas por respaldo.
- **Objetivo:** Programar la secuencia de comandos necesaria para guiar al gato hasta la casilla del mineral y realizar la acción de picar.
- **Ejemplo de solución:** `Avanzar` - `Girar der.` - `Avanzar ×5` - `Picar`.
- **Ejecución y reinicio:** Cada ejecución inicia desde la posición original, al presionar R el nivel se restablece a su estado inicial conservando intacto el programa construido por el usuario.

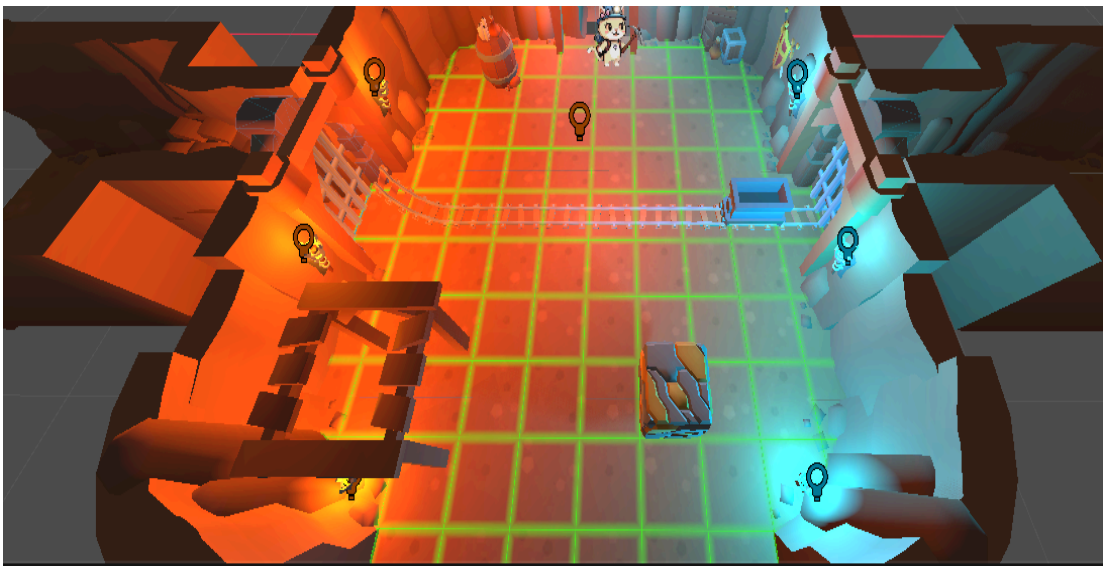
  Para ello el proceso de elaboración del juego comenzó por recopilar assets 3D para dar uso de MeshLibrary para Godot . Esto permite mediante cuadrículas (malla de nxn) agregar los modelos 3d de manera más sencilla para la creación de un nivel.

  Decidimos implementar un primer mapa para implementar el nivel tutorial, usando bloques simples. En la imagen adjunta se observa como queda construido el primer nivel.



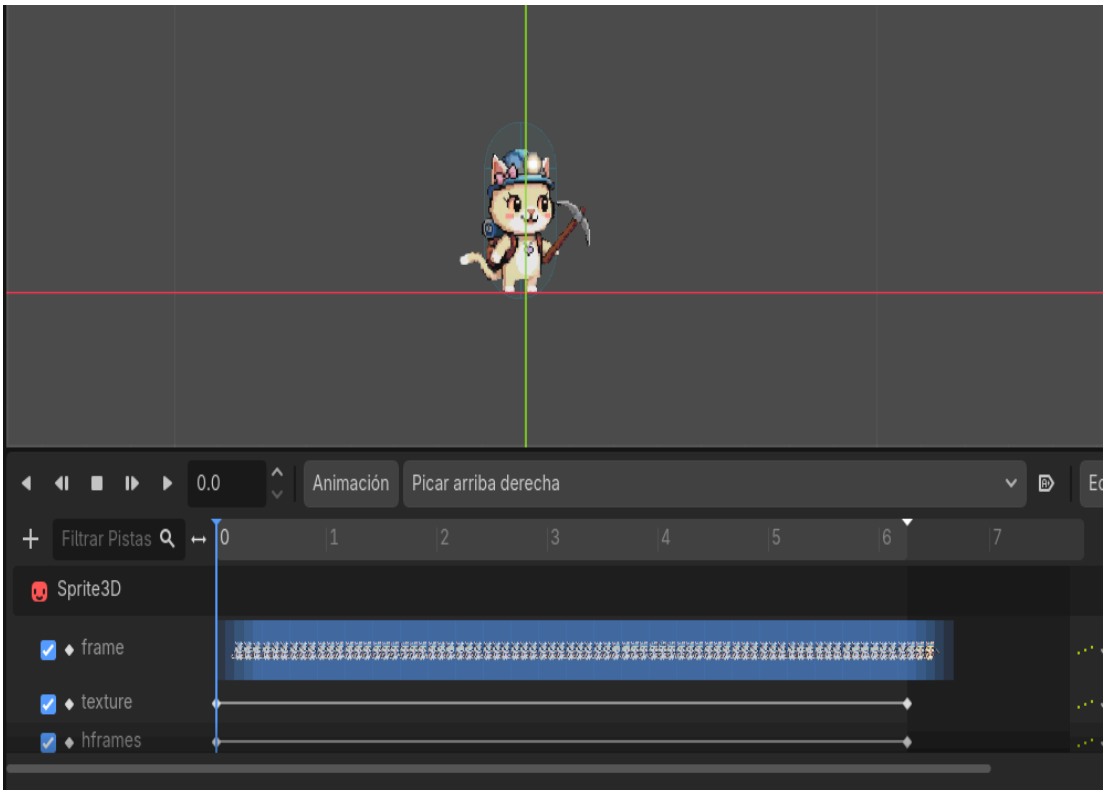
  En este mundo colocamos un bloque gris/dorado que representa la meta del nivel el cual finalizará en el momento que el bloque sea picado.

  Ahora para que los usuarios pudieran ver claramente cómo podía moverse en el mundo, se implementó una cuadrícula en el piso para que se sepa claramente a qué celdas puede desplazarse.

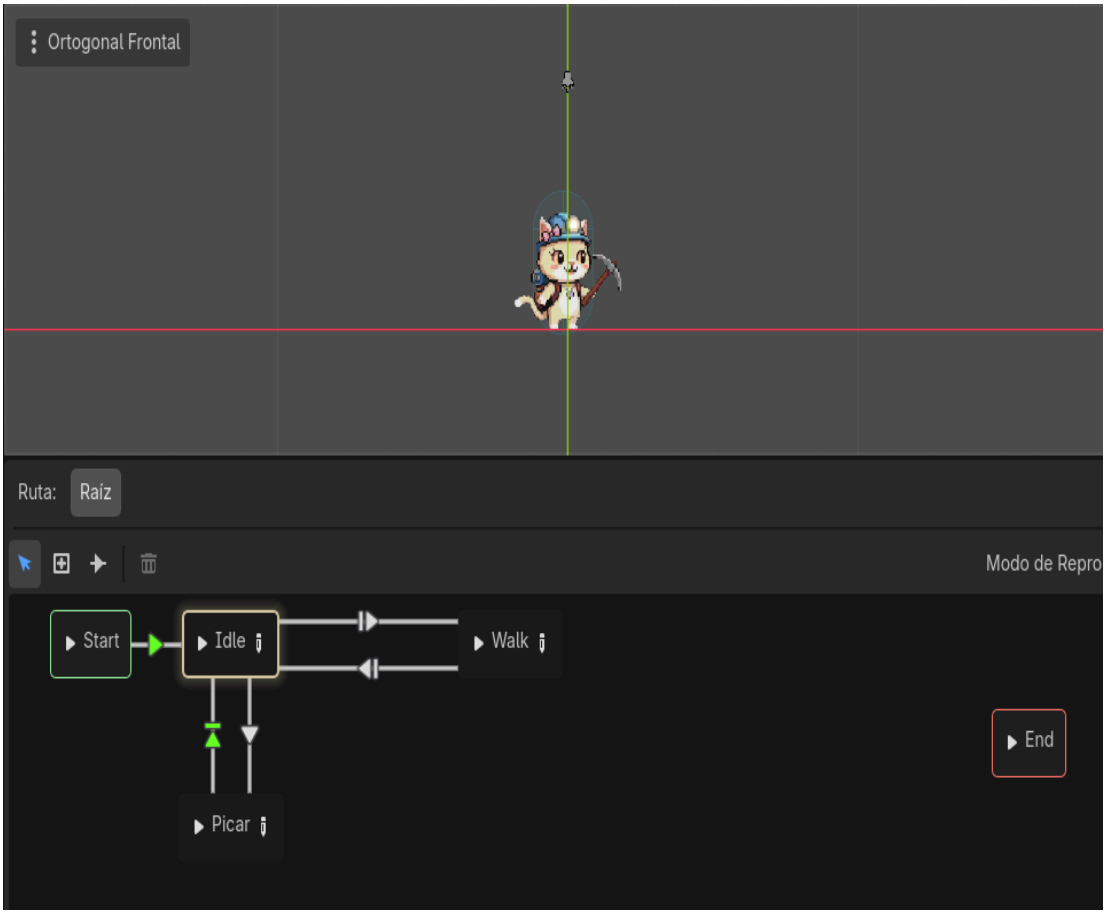


Esta malla se puede configurar de diferentes colores y tiene un efecto de parpadeo para que se pueda dar suficiente contraste para que los usuarios lo vean claramente.

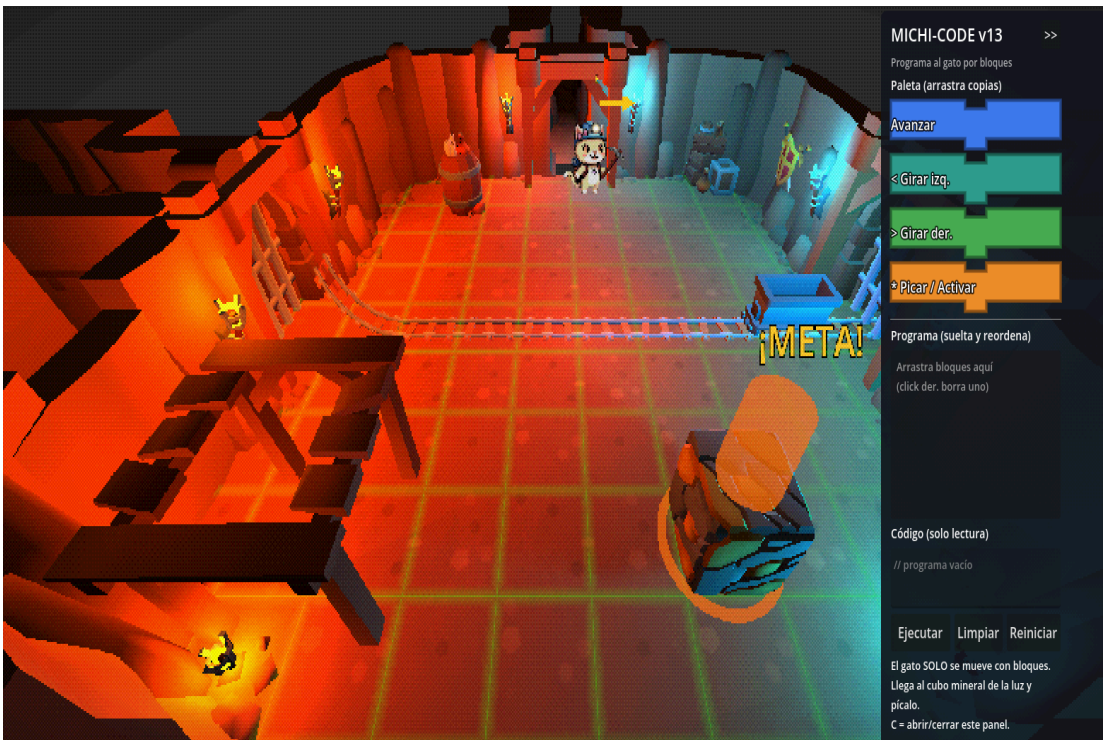
Ahora, se trabajaron en animaciones básicas de movimiento , tales como caminar a la derecha, a la izquierda, posiciones de reposo y de picar, usando AnimationPlayer



y se configura una máquina de estados para las animaciones, usando un animationTree, configurando las animaciones para caminar, reposo y picar (para este avance se trabajaron estas animaciones solamente)



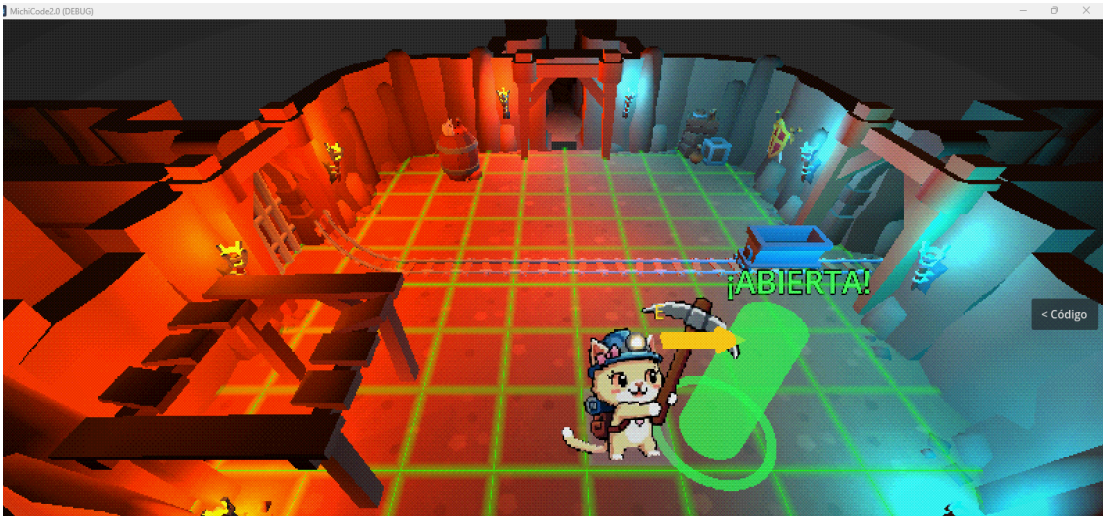

Una vez creadas y configuradas las animaciones se hace una implementación de los bloques gráficos y se establece la condición de victoria que radica en picar el mineral.




Los bloques son funcionales y las animaciones de movimiento trabajan correctamente.
Adicionalmente se configuró un controlador de cámara para que el desplazamiento del jugador sea más evidente.




y se colocó una flecha en la parte superior para que el jugador sepa hacia que dirección se desplaza.
y cuando el jugador logra minar el bloque se abre una puerta que permitirá cambiar de mapa en la próxima entrega.


MichiCode2.0 (DEBUG)
¡ABIERTA!
< Código